



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING**

**A
MINOR PROJECT REPORT
(MID TERM DEFENSE)
ON
CUSTOMER SEGMENTATION AND INTELLIGENT
MOBILE
RECOMMENDATION SYSTEM**

SUBMITTED BY:

AAYUSH CHHUKA	(NCE080BCT002)
HOM RAJ BHANDARI	(NCE080BCT016)
SUDIP KUMAR TAMANG	(NCE080BCT042)
SURESH KHADKA	(NCE080BCT046)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING
LALITPUR, NEPAL

FEBRUARY, 2026

Acknowledgements

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering at National College of Engineering for providing the resources and guidance necessary to undertake this project.

We extend our heartfelt thanks to our project supervisor for their invaluable mentorship, constructive feedback, and continuous encouragement throughout the project proposal phase. Their expertise in machine learning and data engineering has been very helpful in shaping the direction of this work.

We also acknowledge the open-source communities behind XGBoost, scikit-learn, pandas, Node.js, Express.js, Prisma, React, and the Kaggle platform for making high-quality datasets and tools freely accessible to researchers and students worldwide.

Finally, we thank our families and fellow students for their moral support and patience during the preparation of this proposal.

Aayush Chhuka

NCE080BCT002

Hom Raj Bhandari

NCE080BCT016

Sudip Kumar Tamang

NCE080BCT042

Suresh Khadka

NCE080BCT046

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
BCT	Bachelor in Computer Engineering
CPU	Central Processing Unit
CSS	Cascading Style Sheet
CSV	Comma Separated Values
GBM	Gradient Boosting Machine
HTML	HyperText Markup Language
ID	Identifier
JSON	JavaScript Object Notation
ML	Machine Learning
MP	Megapixel
ORM	Object Relational Mapper
OTP	One Time Password
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
UX	User Experience
XGBoost	Extreme Gradient Boosting

1. Introduction

1.1 Background

Customer segmentation is the process of grouping buyers based on shared traits to better understand their needs. While traditional methods rely on basic details like age or income, these often miss the deeper preferences of modern mobile phone users. A customer's choice is now driven more by their "tech tier" (performance needs) and brand loyalty than by simple demographics.

Recent progress in machine learning (ML) allows us to find these hidden patterns automatically by looking at past sales and phone specifications. This project plans to use the XGBoost (Extreme Gradient Boosting) model, which is a powerful tool known for its high accuracy with organized data. Unlike older methods, it is excellent at handling complex categories, such as specific phone chipsets or locations, without needing manual data preparation [?].

Current market conditions reveal two main gaps in mobile retail. First, most recommendation systems need a lot of user history to work. This causes a "cold-start problem" where the system cannot help new users or suggest new products [?]. Second, there is a lack of simple tools to help marketing teams match products to the right audience. This often forces them to rely on slow and inaccurate guesswork.

This report describes the work completed so far on the integrated ML pipeline for personalised mobile retail. As of this mid term defense, the data collection and preprocessing phase has been completed, the backend service layer has been implemented in Node.js, and a first version of the web frontend has been built. The machine learning training, model serving, and the full recommendation flows are scheduled for the next project phase.

1.2 Problem Statement

The mobile retail industry faces three major problems when trying to match the right phones with the right customers. First, retailers do not clearly understand what customers really want. They usually group people only by age or location, which does not show their real interests, so customers often receive advertisements for products they do not need. Second, recommendation systems need past purchase or browsing history to give suggestions. This becomes a problem for new customers or newly launched phones because there is no previous data, so helpful recommendations cannot be provided.

Third, even when retailers have customer data, they do not have a simple system to connect this data with marketing during a new product launch. Because of this, marketing

teams depend on guesswork, which is slow and often not accurate.

The first phase of this project, completed before the current development cycle, focused on gathering the raw data sources and producing a clean combined dataset. The second phase, which is the subject of this report, focuses on building the supporting platform: a Node.js backend with secure user authentication, a relational database, and a React based frontend. The recommendation engine itself will be built on top of this foundation in the next phase.

1.3 Objectives

The objectives listed below describe the work that has been delivered so far or that is currently in progress. Items already implemented are clearly identified.

- I. **Completed.** To collect and preprocess the three required datasets: mobile phone specifications from Kaggle, AnTuTu benchmark scores obtained through web scraping from NanoReview, and a large e-commerce mobile sales dataset from Kaggle. The three sources have been merged into a single combined dataset keyed on processor and phone model, ready for model training.
- II. **Completed.** To design and implement a secure authentication backend using Node.js with Express.js. The system supports user registration, login, logout, password hashing with bcrypt, OTP based email verification, OTP based password reset, and session management backed by PostgreSQL.
- III. **Completed.** To set up a relational database using PostgreSQL with the Prisma ORM. The schema includes users, OTP records, and migration scripts, and provides referential integrity through foreign keys between users and their OTPs.
- IV. **Completed.** To provide input validation for all authentication endpoints using express-validator, with password rules, email format checks, phone number checks, and duplicate detection for email and phone fields.
- V. **In Progress.** To develop a brand recommendation system that suggests suitable mobile brands for customers based on purchasing trends in different locations.
- VI. **In Progress.** To classify customers into different technology preference groups, ranging from high-end users to budget buyers, based on the performance level of the phones they purchase.
- VII. **In Progress.** To analyze customer camera preferences by examining the camera specifications of previously purchased phones and identifying whether they prefer selfies, professional photography, or general use.
- VIII. **In Progress.** To design a system that matches newly launched phones with the

most suitable customer groups, even if there is no previous sales data available.

- IX. **In Progress.** To identify brand-loyal and brand-switching customers by analyzing their purchase history over time.
- X. **Completed.** To integrate all features into a simple and user-friendly web application. The React based frontend at this stage includes the sign in page, multi step registration with OTP verification, password reset screen, and a placeholder dashboard showing future customer segment and camera preference views.

1.4 Scope

This project works in the mobile phone retail field and includes the following scope:

- I. It uses three real-world datasets including phone specifications, benchmark scores, and large-scale online purchase history data for analysis and modeling. These datasets have already been collected, cleaned, and merged in the earlier phase of the project.
- II. It develops machine learning models using XGBoost to segment customers and recommend products based on location, behavior, technical features, and purchase patterns.
- III. It builds a Node.js backend service using Express.js and the Prisma ORM on top of PostgreSQL. The backend handles user management, OTP verification, sessions, and exposes a placeholder route for future product endpoints.
- IV. It creates a React based web application with HTML, CSS, and JavaScript to provide the sign in, registration, OTP, password reset, and dashboard pages used by customers and retailers.

The limitations of this project are:

- I. It does not include real-time stock management or automatic price optimization.
- II. It does not include a native mobile application for Android or iOS, and works only as a web system.
- III. It does not connect directly with any live e-commerce website or payment system.
- IV. It only uses the available open datasets and does not include additional private customer data.
- V. The machine learning models and the recommendation endpoints are not yet integrated with the running application. The current build of the system delivers the data foundation, the authentication platform, and the user interface, while the trained models and serving layer are planned for the next phase.

2. Literature Review

2.1 Related Work

Chen and Guestrin [?] introduced XGBoost, a scalable tree boosting system that has become a benchmark for tabular data. Its regularized learning objective and efficient handling of mixed features make it ideal for customer segmentation tasks, directly supporting its use as the sole algorithm in this project.

Singh [?] developed a customer segmentation and recommendation system with a similar multi-dimensional approach, categorizing users into segments like Tech Savvy, Brand Lovers, and Camera Lovers based on purchase behavior. Their work, publicly available on GitHub, demonstrates a practical application of micro-segmentation for mobile phone recommendations using machine learning and a Flask backend. This project serves as a key reference for both the segmentation logic and the overall system architecture.

Qi et al. [?] proposed an XGBoost recommendation algorithm integrated with collaborative filtering to specifically address the cold-start problem. By using XGBoost to predict item preferences for new users, their work validates XGBoost's capability to generate recommendations without historical purchase data, a core requirement for this project's customer recommendation feature.

Ayyappan et al. [?] demonstrated a hybrid BiLSTM-XGBoost model for purchase intent prediction. While their work shows XGBoost's effectiveness as a final classifier, this project focuses on a pure XGBoost approach for all segmentation and regression tasks, proving its standalone power for structured, tabular phone specifications and transaction data.

2.2 Related Theory

2.2.1 Gradient Boosting Machines

Gradient Boosting is a machine learning technique that builds a strong predictive model by combining many simple models in a sequential manner. The core idea is to train new models that correct the mistakes made by previously trained models. Each new model focuses on the difficult cases that earlier models could not handle correctly, gradually improving the overall prediction accuracy.

The process begins with a simple initial prediction, often just the average value of the target variable. In each subsequent step, a new model is trained on the errors or residuals of the combined previous models. These residuals represent the gap between the actual

values and the predictions made so far. By learning to predict these residuals, each new model helps close this gap, pushing the ensemble toward better performance.

2.2.2 XGBoost Overview

XGBoost, which stands for Extreme Gradient Boosting, is an advanced implementation of gradient boosting that has become one of the most popular and successful machine learning algorithms. It was designed with both speed and accuracy in mind, making it suitable for a wide range of prediction tasks.

The algorithm builds an ensemble of decision trees one after another, where each tree learns from the mistakes of all previous trees. Unlike a random forest where trees are built independently, the trees in XGBoost work together as a team, with each new member focusing on what the team got wrong. This sequential learning approach allows the model to capture complex patterns in the data that simpler models might miss.

2.2.3 Mathematical Foundation

The learning process in XGBoost is guided by a carefully designed objective function that balances two competing goals. The first goal is to make accurate predictions, measured by a loss function that quantifies the difference between predicted and actual values. The second goal is to keep the model simple and general, measured by a regularization term that penalizes overly complex trees.

The overall objective that XGBoost minimizes during training can be expressed as:

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (2.1)$$

where:

- $\mathcal{L}(\theta)$ is the overall objective function that XGBoost minimizes during training. It represents the total cost that balances prediction errors against model complexity.
- n is the total number of training samples in the dataset.
- y_i is the actual true value for the i -th training sample.
- $\hat{y}_i$ is the predicted value produced by the model for the i -th training sample.
- $l(y_i, \hat{y}_i)$ is the loss function that measures the difference between the actual value and the predicted value for a single sample. For regression tasks this is often squared error, and for classification tasks it is often logarithmic loss.
- K is the total number of decision trees in the ensemble.
- f_k represents the k -th decision tree in the ensemble.

- $\Omega(f_k)$ is the regularization term that penalizes the complexity of the k -th tree to prevent overfitting.

In this equation, the first term sums the loss across all n training samples, where l measures how wrong the prediction is for each sample. The loss function can be chosen based on the task, such as squared error for regression problems or logistic loss for classification. The second term adds up the complexity penalty for all K trees in the ensemble. This penalty term takes the form:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (2.2)$$

Here T represents the number of leaves in the tree, and w represents the scores assigned to each leaf. The gamma parameter controls how much the model is penalized for adding more leaves, helping to prevent the tree from growing too deep and memorizing noise in the data. The lambda parameter controls the magnitude of the leaf scores, encouraging the model to make more conservative predictions.

2.2.4 Key Technical Features

XGBoost introduces several innovations that contribute to its strong performance. One important feature is the use of second order derivatives during tree construction. While basic gradient boosting only uses the first derivative of the loss function, XGBoost also uses the second derivative, which provides more information about the shape of the loss surface. This allows the algorithm to make better decisions about how to split the data at each tree node.

Another crucial feature is the built in handling of missing values. In real world datasets, missing values are common and can be problematic for many algorithms. XGBoost learns the best direction to send missing values during training, effectively letting the data tell the algorithm how to handle gaps. This eliminates the need for separate imputation steps and often leads to better models.

Regularization is deeply integrated into XGBoost, going beyond simple pruning of trees after they are built. Both L1 and L2 regularization are applied directly to the leaf scores, which helps prevent individual trees from becoming too confident in their predictions. This regularization, combined with techniques like column subsampling where each tree only sees a random subset of features, helps the ensemble generalize well to new data.

2.2.5 Interpretability Considerations

While XGBoost models are more complex than single decision trees, they still offer ways to understand what the model has learned. Feature importance scores can be calculated to show which input variables had the biggest impact on predictions. For our camera preference model, this could reveal whether front camera resolution or rear camera resolution matters more for classifying customers into different camera preference groups.

The algorithm also provides tools for understanding how individual predictions are made. By examining the path a sample takes through each tree in the ensemble, we can see which features contributed to the final decision. This transparency is valuable when we need to explain why a particular phone was recommended to a customer or why a specific customer was assigned to a certain audience segment.

2.2.6 Why XGBoost for This Project

For this project, XGBoost handles the mixed data types present in our phone specifications and transaction records without requiring extensive preprocessing. The algorithm is robust to outliers in features like price and benchmark scores, which might otherwise skew the results. Its built in regularization helps ensure that our models will perform well on new customers and phones they have not seen during training.

The sequential learning nature of gradient boosting means XGBoost can focus on the difficult cases in our data. In customer segmentation, some customers may have clear preferences that are easy to predict, while others have more complex behavior patterns. XGBoost will naturally allocate more of its capacity to learning these challenging cases, potentially improving the quality of recommendations for all customers.

Finally, XGBoost has a proven track record in both academic research and industry applications involving tabular data. Its consistent performance across many different types of problems makes it a reliable choice for implementing all the models in our segmentation pipeline, from tech tier classification to brand loyalty prediction.

2.3 Website Development for End Users

2.3.1 User Interface Design

The website features a clean interface designed for two distinct user types: retailers and individual customers. The homepage presents clear navigation paths for both user groups, with a retail dashboard for business users and a customer portal for individual phone shoppers. The design prioritizes simplicity, ensuring that users can easily access the recommendation features without technical expertise.

2.3.2 Retailer Functionality

Retailers will access a dedicated dashboard by logging into their accounts. The main feature allows them to enter a location and receive instant category predictions for phones popular in that area. The system will display top mobile phones along with their specifications, camera tier classifications, and tech tier designations. This helps retailers understand which phones to stock based on local customer preferences and target their marketing efforts toward specific audience segments. As of the mid term defense, a placeholder retailer dashboard layout has been built in the React frontend to demonstrate the future visual structure and information hierarchy. The actual prediction features will be connected in the next phase.

2.3.3 Customer Functionality

Individual customers can register as new users or log in with their existing account. New users are guided through a simple registration process where they enter their full name, email, password, and confirm password. After registration, a six digit OTP is sent to the email address for verification. The current React frontend already supports the multi step registration, the OTP entry screen with paste support and a resend cooldown timer, and the password reset flow. Returning users can sign in with their email and password.

2.3.4 Recommendation Display

Once the recommendation models are integrated in the next project phase, the website will present recommendations in an easy to scan format showing top matching phones with their key specifications. Each recommended phone will include its camera tier classification, tech tier designation, and a match score explaining why it suits the user's preferences. Customers will be able to compare multiple options and explore detailed specifications to make informed purchasing decisions. The current React dashboard already reserves visual space for these elements and shows a tech tier distribution and camera preference distribution with placeholder values, providing a clear target for the next phase of work.

3. Proposed Methodology

3.1 Feasibility Study

The feasibility study evaluates whether the proposed customer segmentation and mobile recommendation system can be realistically developed and deployed under existing constraints.

The project flow follows standard SDLC phases from planning and analysis through implementation, testing, and deployment.

3.1.1 Technical Feasibility

The proposed system is technically feasible given the team's proficiency and access to essential tools and technologies.

Technology Stack:

- Frontend: React based single page application using HTML, CSS, and JavaScript, with Vite compatible tooling through Create React App, Axios style fetch calls, and Bootstrap friendly component structure for responsive layout.
- Backend: Node.js with the Express.js framework for server side logic, including routing, session management, and middleware based authentication.
- Database: PostgreSQL accessed through the Prisma ORM, providing typed schema definitions, automatic migrations, and referential integrity for users and OTP records.
- Machine Learning: Python with XGBoost, pandas, and scikit-learn for the data preprocessing and segmentation models. The trained models will be served to the Node.js backend through a small HTTP bridge.
- Version Control: Git and GitHub for collaborative development and code management.
- Runtime: Node.js for the backend, npm for package management, nodemon for development, and the React development server for the frontend.

Team Skills: The development team has foundational skills in JavaScript and TypeScript for the application layer, basic Python for data work, SQL and relational database concepts, and introductory machine learning. The team has experience with REST APIs, JSON based request response flows, and modern frontend component design.

Development Tools: VS Code for code editing, Google Colab for exploratory data

analysis, Git and GitHub for version control, Postman for API testing, pgAdmin for the PostgreSQL database, and the nodemon development server for the Node.js backend.

3.1.2 Economic Feasibility

The proposed system is economically feasible as it relies entirely on open source tools and free development platforms. The Node.js runtime, Express.js framework, Prisma ORM, PostgreSQL database, React library, XGBoost library, pandas, and scikit-learn are all freely available with no licensing costs.

Cost assumptions are based on academic project requirements and free tier boundaries of selected services.

Table I: Economic Feasibility Estimation

Item	Estimated Cost
Development Tools	Rs. 0 (open source)
JavaScript Libraries	Rs. 0 (React, Express, Prisma all open source)
Python Libraries	Rs. 0 (XGBoost, pandas, numpy, scikit-learn all open source)
Machine Learning Models	Rs. 0 (no licensing costs for model development)
Dataset Acquisition	Rs. 0 (Kaggle public datasets)
Domain Name	Rs. 0 (free for academic demonstration)
Web Hosting	Rs. 0 (local hosting or free tier cloud)
Miscellaneous	Rs. 500-1,000
Total Initial Cost	Rs. 500-1,000

For academic demonstration purposes, local hosting on development servers will suffice. Commercial deployment would require cloud hosting and domain registration expenses as the user base grows.

3.1.3 Operational Feasibility

The system is operationally feasible as it aligns well with the team's development capabilities and workflow. The web-based platform can be accessed through any modern browser on desktop and mobile devices, eliminating the need for platform-specific development. The interface is designed to be intuitive for both retailers seeking market insights and customers looking for personalized phone recommendations.

The platform requires minimal training for end users. Retailers can easily enter locations and view category predictions, while customers can register their preferences and receive recommendations with just a few clicks. The simple two-pathway design ensures high usability for both technical and non-technical users.

3.1.4 Legal and Ethical Feasibility

- I. All user data used in this project is sourced from publicly available datasets on Kaggle, which contain no personally identifiable information.
- II. The phone specifications dataset, Antutu benchmark scores, and eCommerce purchase transactions are all obtained from Kaggle's public repositories [?] and [?] and web scraping from [?] for academic research and educational purposes.
- III. Customer purchase history and preferences used for training the XGBoost models are derived from anonymized Kaggle datasets with all identifying information removed.
- IV. The project follows ethical guidelines for machine learning projects including transparency in data sources, responsible use of public datasets, and prevention of any potential misuse.
- V. As an academic project, the system is not intended for commercial deployment and will not process real user data beyond controlled testing with synthetic information.
- VI. Location information in the datasets is used solely for geographic brand recommendation models and exists only at city level with no individual address details.
- VII. All machine learning models are trained exclusively on Kaggle's publicly available datasets, ensuring compliance with data usage terms and avoiding any privacy concerns.
- VIII. The project acknowledges and follows Kaggle's dataset terms of use, which permit academic and research applications of the provided data.
- IX. No proprietary or confidential information is used in model training, and all datasets are properly credited to their original sources on Kaggle.

3.2 Requirement Analysis

3.2.1 Functional Requirements

The system allows retailers and customers to register using email and password, and create profiles with basic details and location. Retailers can use a dashboard to enter a location and get predictions of popular phones in that area, along with complete specifications, camera and tech category insights, and audience trends for better inventory planning. Customers can set preferences for camera quality, performance, and budget, or use customer ID to access personalized recommendations with match reasons and comparison options. The platform also classifies users by tech level, camera preference, regional brand popularity, and brand loyalty behavior. For newly launched phones, it

predicts the most suitable audience by combining multiple model outputs.

3.2.2 Non Functional Requirements

The system is designed to be fast, easy to use, secure, reliable, and scalable. Model predictions are expected to return quickly, pages should load smoothly, and users should get search results without delay even when many people use the platform at the same time. The interface should be simple and clear for both technical and non technical users, with consistent design, easy navigation, and helpful error messages. The application should remain available throughout the day with stable uptime and proper handling of temporary backend failures. It should work across major browsers and different screen sizes, while also handling slow internet connections without crashes or session loss. The architecture should support future growth in users and data, allow new machine learning models to be added easily, and handle large scale batch predictions. The codebase should remain maintainable through modular design, coding standards, documentation, and independent model retraining. Reliability is supported by proper input validation, structured error responses, prediction confidence indicators, and detailed logging for monitoring and debugging.

3.3 Proposed System Design

3.3.1 System Architecture

The Customer Segmentation and Intelligent Mobile Recommendation System follows a modular three tier architecture with clear separation between data, model, and application layers. The frontend is developed as a React based single page application that communicates with the backend over JSON HTTP. The Node.js backend, built on the Express.js framework, handles HTTP routing, user authentication, session management, and will orchestrate calls to the machine learning models in the next project phase. The model layer consists of XGBoost models trained in Python that perform the core segmentation and recommendation tasks. PostgreSQL, accessed through the Prisma ORM, serves as the relational database for storing user profiles, OTP records, and future tables for phone specifications, customer preferences, and recommendation history. The architecture ensures that each component can be developed, tested, and maintained independently while working together through well defined APIs.

3.3.2 System Flowchart

The system flowchart illustrates the complete workflow for both retailer and customer interactions with the platform. The process begins at the start node and branches into two distinct pathways based on user type, ensuring each user group follows a tailored flow suited to their specific needs.

Retailer Pathway: Retailers will access the category prediction feature by first enter-

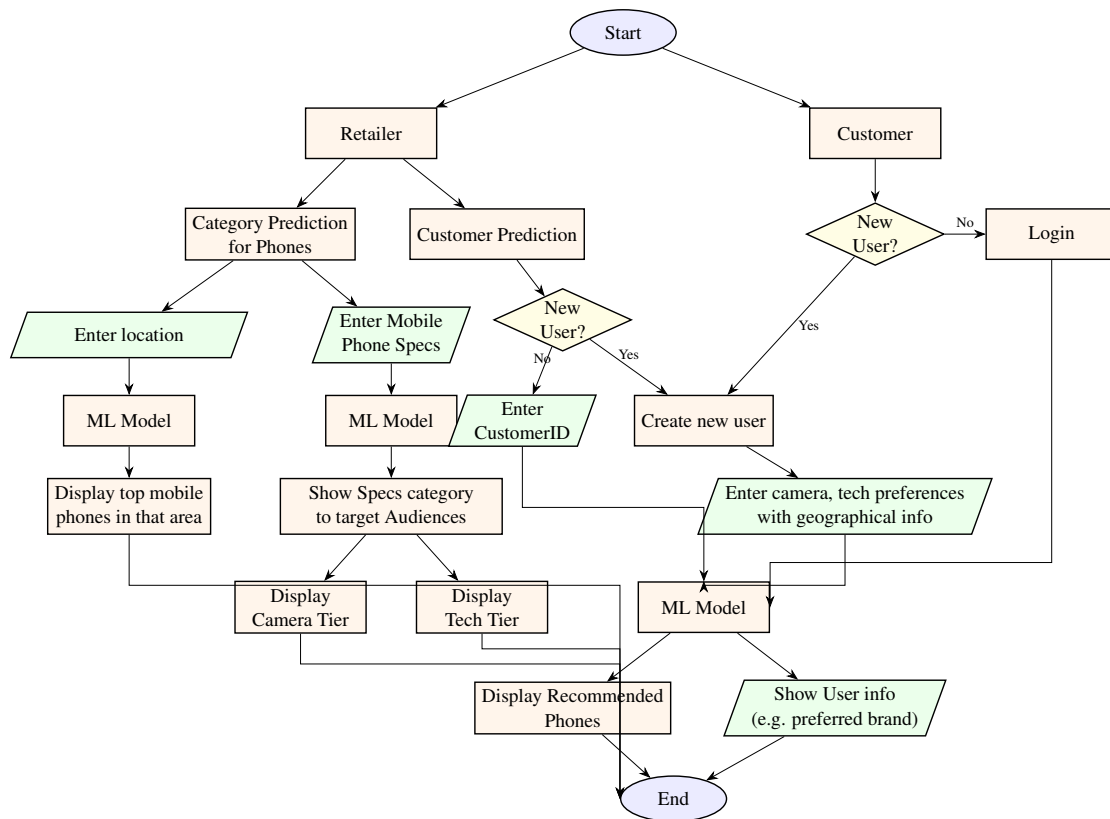


Figure I: System Flowchart Showing Retailer and Customer Workflows

ing a location of interest, such as a specific city or area where they operate or plan to launch marketing campaigns. They will then provide mobile phone specifications including brand, processor, RAM, storage, and camera details. This input will be processed by the machine learning model, which will analyze historical purchase data and geographic trends to generate predictions about phone popularity in that specific area. The system will then display the top mobile phones recommended for that location, along with specification categories that help retailers understand which audience segments to target. Additional outputs will include camera tier classifications showing whether phones appeal to selfie focused users, photography enthusiasts, or balanced users, and tech tier designations indicating performance expectations from entry level to flagship enthusiasts. These comprehensive insights will empower retailers to make data driven inventory decisions, optimize marketing campaigns for specific customer segments, and allocate stock strategically across different store locations based on local preferences.

Customer Pathway: Customers enter the customer prediction flow where the system first checks whether they are a new or existing user through a conditional branching node. New users are directed to a registration process in the React frontend where they create an account by entering their full name, email, password, and confirm password. After registration, a six digit OTP is generated by the Node.js backend and emailed

through Nodemailer. The user enters the OTP in the frontend, and the backend verifies it against the active record in the `otps` table. Returning users can sign in with their email and password. Once authenticated, all customer inputs regardless of user type will be processed by the machine learning model in the next phase, which will apply the trained XGBoost algorithms to analyze preferences against the phone specifications database. The system will then generate personalized phone recommendations ranked by match scores. The display interface will show recommended phones along with detailed user information including preferred brands predicted by the geographic model, camera tiers matching their photography preferences, and tech tiers aligned with their performance expectations. Additional context such as confidence scores for borderline predictions and explanations of why each phone was selected will help customers make informed decisions. The workflow concludes at the end node after successfully displaying all relevant outputs to both user types, completing the full system interaction cycle.

3.3.3 Data Preparation and Machine Learning Workflow

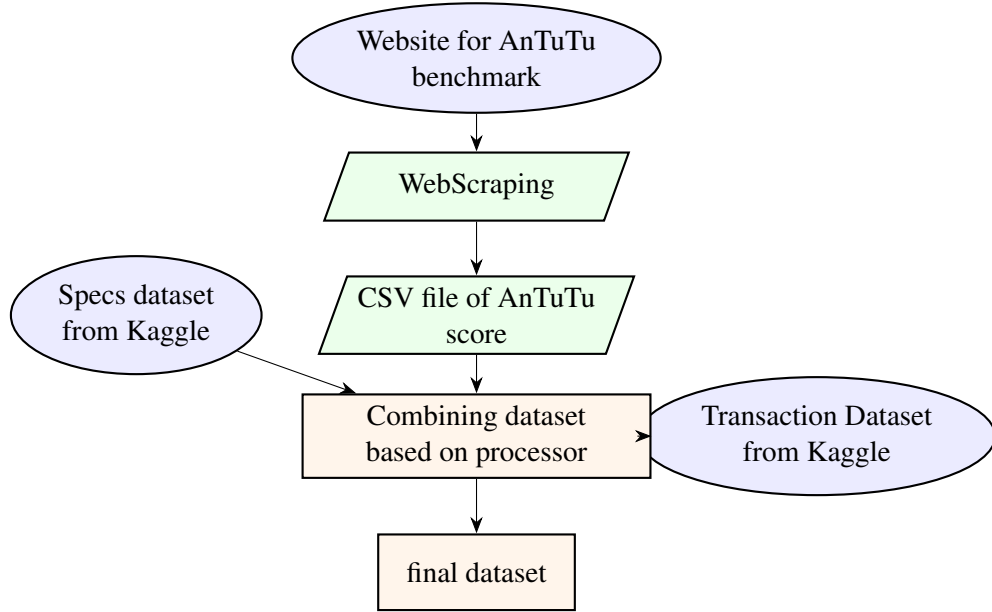


Figure II: Data Collection and Integration Process

The data collection process begins with three primary sources. Phone specifications are obtained from a Kaggle dataset, providing comprehensive hardware details including processor, RAM, storage, and camera information. AnTuTu benchmark scores are acquired through web scraping from the NanoReview website, resulting in a CSV file containing performance scores for various chipsets. Transaction data including customer purchase history is sourced from a Kaggle eCommerce dataset. These three datasets have been merged based on processor and phone model information to create a final integrated dataset that links hardware specifications with performance scores and market transactions. This data preparation step was completed in the first week of

the project before the development phase began, and the merged dataset will serve as the foundation for all subsequent model training and analysis in the next phase.

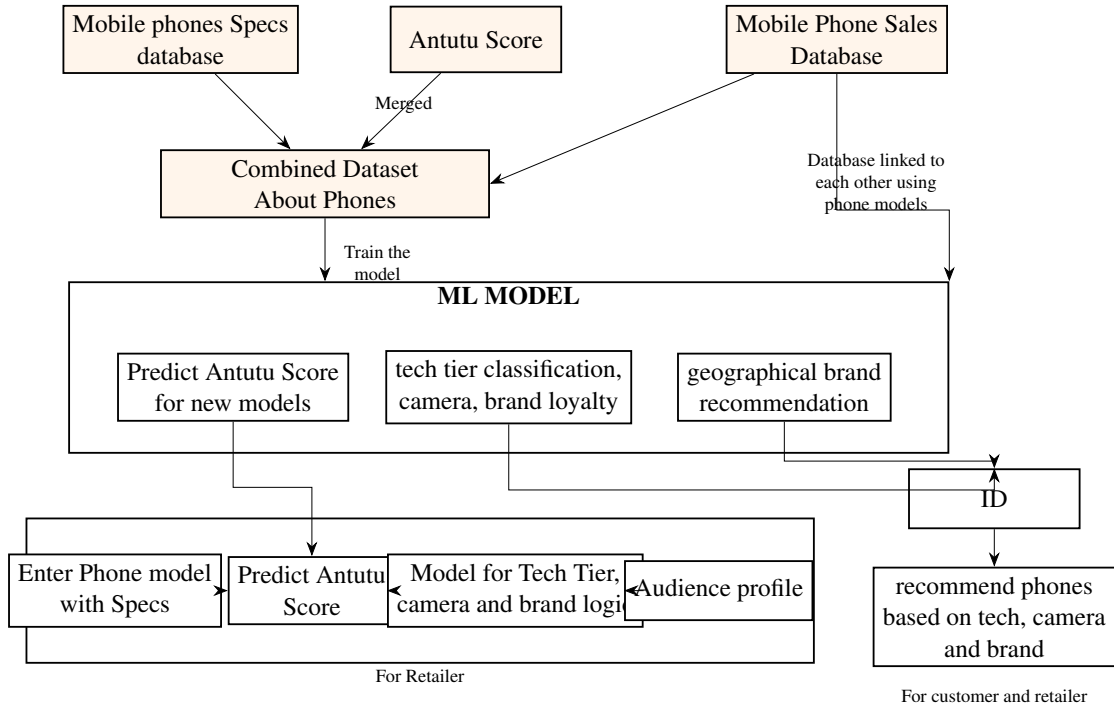


Figure III: Machine Learning Model Workflow

The machine learning workflow illustrates how the integrated dataset is used to train a unified XGBoost model. The mobile phone specifications database, Antutu scores, and mobile phone sales database are merged, creating a combined dataset about phones. This dataset is used to train the machine learning model, which learns to predict Antutu scores for new phone models based on their specifications. The model also performs tech tier classification, camera preference analysis, brand loyalty identification, and geographic brand recommendation. When a retailer enters a phone model with specifications, the system predicts its Antutu score and applies tech tier, camera, and brand logic to generate a complete audience profile. For customer facing recommendations, the system takes customer ID as input and recommends phones based on matching tech tier, camera preferences, and brand affinity, serving both retailer and customer needs through an integrated pipeline.

3.4 Implementation Details

3.4.1 Frontend Development

The frontend is built as a React based single page application using HTML, CSS, and JavaScript, providing a responsive web interface accessible on desktop and mobile browsers. The system at this stage includes the authentication pages with sign in and multi step registration forms for retailers and customers, plus a placeholder dash-

board that demonstrates the future retailer dashboard layout, including summary statistic cards, tech tier distribution, camera preference distribution, and a recent activity feed.

The multi step registration flow collects the user's full name, email, password, and confirm password in step one, then displays a six digit OTP entry screen in step two. The OTP screen supports paste from the clipboard, automatic focus movement between digits, backspace navigation, a resend cooldown timer, and an error message when the code is incomplete. The sign in page includes a password show or hide toggle, a remember me checkbox that stores the email in local storage, and a link to the password reset page.

A reusable toast notification component is provided to surface success and error messages returned by the backend. All visible labels, layout, and visual structure have been reviewed against the proposal so that the recommendation cards, match score badges, camera tier chips, and tech tier labels planned for the next phase can be added without breaking the layout.

The frontend communicates with the Node.js backend through JSON HTTP requests. Bootstrap compatible utility classes are used for layout in several places. JavaScript handles dynamic content updates and form validation without page reloads.

3.4.2 Backend Development

The backend of the system is built using Node.js with the Express.js framework, which provides a lightweight and scalable foundation for the application. Express handles HTTP requests, URL routing, user authentication, and session management, and the application makes use of standard Node.js middleware for cookies, body parsing, sessions, and error handling. The system uses a PostgreSQL relational database accessed through the Prisma ORM, which gives typed schema definitions, automatic migrations, and referential integrity for the users and OTP records.

The authentication system is based on Passport Local with a custom strategy that accepts email and password. Passwords are hashed with bcrypt at registration time and compared with the same library at login. Sessions are stored server side in the PostgreSQL session table through `connect-pg-simple` so that multiple backend instances can share a session store if needed in the future. OTP codes are generated with the Node.js `crypto` module, sent to the user through Nodemailer using SMTP credentials read from environment variables, and stored in the `otps` table together with expiry and used flags. A transactional update is used when an OTP is verified or when a new one is issued, so that no two OTPs for the same user are simultaneously active.

The backend follows a modular structure with separate folders for routes, controllers, services, middleware, validation, configuration, and database access. Database models

are defined in the Prisma schema. Controllers contain the request handlers and orchestrate calls into the services. Services contain the reusable business logic such as registration, OTP verification, login, logout, and password reset.

3.4.3 Machine Learning Model Integration

The machine learning model layer is implemented using Python with XGBoost, pandas, and scikit-learn. The data preparation and exploration work, including web scraping the AnTuTu benchmark scores and merging the three datasets, was completed in the first week of the project. The training scripts, model artefacts, and the bridge to the Node.js backend are scheduled for the next phase.

The plan for the next phase is as follows. The system will use a set of XGBoost based models trained on the merged dataset combining mobile phone specifications, AnTuTu benchmark scores, and mobile phone sales data. The database will link all information through phone models, enabling comprehensive analysis. When a retailer enters a phone model with specifications, the model will predict its AnTuTu score and then determine the appropriate tech tier classification, camera preference category, brand loyalty pattern, and geographic brand recommendation to generate a complete audience profile. For customer facing recommendations, the system will take customer ID as input and recommend phones based on matching tech tier, camera preferences, and brand affinity.

Trained models will be serialised with joblib or pickle and exposed to the Node.js backend through a small Python HTTP service. The Node.js backend will call this service through an HTTP client with timeouts and retries. Model versioning will be maintained to enable rollback if newer models underperform.

3.4.4 Database Schema

The PostgreSQL database is accessed through the Prisma ORM. The schema is defined in `prisma/schema.prisma` and version controlled through the migration folder. As of the mid term defense, the schema includes two tables.

- **users.** Stores user accounts with fields for name, email (unique), password hash, optional phone number, an `is_active` flag, and an `is_verified` flag.
- **otps.** Stores one time password records linked to a user through a foreign key. Each record holds the six digit code, the expiry timestamp, an `is_used` flag, and the creation timestamp. The `onDelete: Cascade` rule removes OTPs when their user is removed.

In the next phase the schema will be extended with tables for phone specifications, customer preferences, recommendation history, brand loyalty records, and aggregated

location based brand statistics.

3.4.5 Testing Strategy

Unit Testing:

Unit testing focuses on verifying that individual components and functions work correctly in isolation. The Node.js backend modules for password hashing, OTP generation, validation schemas, and Prisma queries will each receive dedicated unit tests. The React frontend will receive component level tests using React Testing Library. The machine learning model scripts will be exercised against a small fixture dataset so that the training pipeline can be validated without depending on the full combined dataset.

Integration Testing:

Integration testing examines how different parts of the system work together. For the completed authentication flow, this includes testing that registration creates a user and an OTP record, that verification updates both the user and the OTP atomically, that login and logout correctly manage the session, and that the protected product route rejects unauthenticated requests. For the next phase, this will include testing the connection between Node.js and the Python XGBoost serving layer, the integration between the React frontend and the Node.js backend, and the integration between the backend and the PostgreSQL database.

User Acceptance Testing:

User acceptance testing involves real users trying out the system to see if it meets their needs. Sample users representing retailers and customers will be asked to test the application and provide feedback. Retailers will test the location based prediction feature and audience insights. Customers will test the registration process, preference setup, and recommendation generation. Their feedback on usability, clarity of results, and overall experience will be collected and used to make improvements.

Performance Testing:

Performance testing checks how well the system responds under different conditions. Response times for location predictions and customer recommendations will be measured to ensure they stay under two seconds. The system will be tested with multiple users making requests at the same time to see if performance degrades. Database query speeds will be monitored as the number of phones and users grows. Testing will also cover how the application performs on different devices and browsers to ensure a consistent experience for all users.

4. Time Schedule

The Customer Segmentation and Intelligent Mobile Recommendation System project is scheduled according to the timeline illustrated in Figure ??.

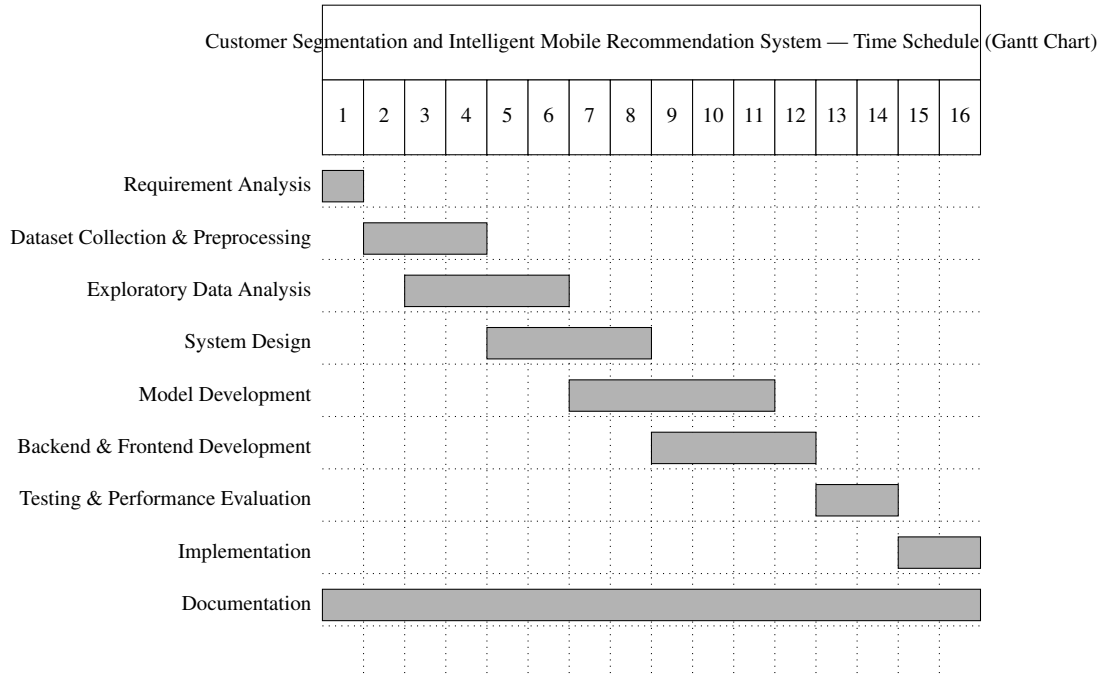


Figure I: Gantt chart showing project timeline for Customer Segmentation and Intelligent Mobile Recommendation System

The project began with data collection and preprocessing in the first week, covering web scraping of AnTuTu benchmark scores and merging the three datasets. Requirement analysis, system design, and exploratory data analysis ran alongside this preparation. Backend and frontend development took place from the second week through the current phase, building the Node.js backend with Express.js and the React based web interface. The documentation has been done from the beginning of the project to the end.

Model development, XGBoost training, model serving, and the integration of the recommendation endpoints are scheduled for the next project phase. Testing and performance evaluation will follow, together with final implementation, deployment, and any remaining documentation work.

5. Expected Output

This chapter is divided into two parts. The first part lists the work that has been completed by the mid term defense and is visible in the current code repository. The second part lists the remaining work and the future enhancements that are scheduled for the next project phase.

5.1 Works Completed

- **Data Collection and Preprocessing.** The three required datasets were acquired and preprocessed during the first week of the project. The Kaggle smartphones specifications dataset, the AnTuTu benchmark scores scraped from NanoReview, and the Kaggle mobile sales dataset were cleaned, normalised, and merged on processor and phone model into a single combined dataset ready for model training.
- **Node.js Backend Service.** A Node.js backend built with Express.js is running locally on port 8001 through nodemon. The service handles JSON request and response bodies, cookies, sessions, and centralised error handling.
- **PostgreSQL Database with Prisma ORM.** A PostgreSQL database is configured and accessed through Prisma. Migration scripts are version controlled in the `prisma/migrations` folder. The schema includes the `users` table and the `otp` table with a foreign key and cascade delete from user to OTP records.
- **User Registration.** Users can register with full name, email, password, confirm password, and an optional phone number. Passwords are hashed with bcrypt before storage. Email and phone uniqueness are enforced through Prisma queries. Validation is performed by express-validator using reusable schema definitions.
- **Email OTP Verification.** After registration, a six digit OTP is generated using the Node.js `crypto` module, sent to the user's email through Nodemailer, and stored in the `otp` table with a five minute expiry. Verification updates the user record to set `is_verified` to true and marks the OTP record as used, both inside a single Prisma transaction.
- **OTP Resend.** A resend endpoint invalidates all outstanding OTP records for the user inside a transaction and issues a fresh code, so that only one active OTP exists at a time.
- **Login and Logout.** Login uses Passport Local with a custom strategy that authenticates against the `users` table. Logout destroys the session on the server

side and clears the session cookie in the browser.

- **Password Reset.** A forget password endpoint accepts the user's email, issues a fresh OTP, and a separate change password endpoint verifies the OTP and updates the password hash, again inside a transaction.
- **Session Management with PostgreSQL Store.** Sessions are stored in the PostgreSQL database through `connect-pg-simple` so that they survive backend restarts and can be shared by multiple instances in the future.
- **Centralised Validation and Error Handling.** A reusable `validationWith` middleware collects express-validator errors and attaches the matched data to the request. A central error handler returns consistent JSON error responses with appropriate HTTP status codes.
- **Authentication Middleware.** An `isAuthenticate` middleware protects selected routes by checking the Passport session. The placeholder product route demonstrates this protection.
- **Basic User CRUD.** Authenticated and unauthenticated user endpoints are provided through `/api/users` for listing, retrieving by id, creating, updating, and deleting users. The id is validated through a dedicated `checkId` middleware that loads the user from the database.
- **React Frontend Authentication Pages.** The React application includes a sign in page with email and password, a multi step registration page with OTP verification, and a password reset page. Each page has client side validation, error messages, loading states, and visual feedback.
- **OTP Entry Interface.** The OTP screen uses a six cell input grid with paste support, automatic focus movement, backspace navigation, a resend cooldown timer, and inline error display.
- **Toast Notifications.** A reusable toast component is included so that the frontend can surface success and error messages returned by the backend.
- **React Frontend Dashboard Layout.** A dashboard page is provided with a top bar, summary statistic cards, a tech tier distribution panel, a camera preference distribution panel, and a recent activity feed. The data is currently hard coded so that the visual structure is ready to receive live data from the recommendation endpoints.

5.2 Remaining Work and Future Enhancements

- **XGBoost Model Training.** Train the five XGBoost models described in the proposal: AnTuTu score regression, tech tier classification with five classes, camera preference classification with three classes, geographic brand recommendation, and brand loyalty classification. Each model will be serialised and stored alongside its metrics file.
- **Python Prediction Service.** Implement a small Python HTTP service that loads the trained models and exposes a `POST /predict` endpoint. The Node.js backend will call this service with timeouts and retries.
- **Recommendation Endpoints.** Add the routes described in the proposal: `POST /api/predict/location`, `POST /api/predict/phone`, and `POST /api/recomm`. The endpoints will return ranked phones with match scores, camera tier, tech tier, and audience insights.
- **Phone and Customer Schema.** Extend the Prisma schema with tables for phone specifications, customer preferences, segments, segment memberships, recommendations, recommendation items, and purchases for loyalty analysis.
- **Frontend Integration.** Connect the React dashboard to the recommendation endpoints so that the hard coded panels are replaced with live data. Add phone cards with match score badges, camera tier chips, tech tier labels, and *why this phone* explanations.
- **Role Based Access Control.** Add a role enum on the user table to distinguish retailers from customers, and protect the recommendation endpoints accordingly.
- **Rate Limiting and Security Hardening.** Add a rate limiter on the authentication endpoints to reduce OTP brute force risk, move OTP state from the session to a database backed token model, and add structured logging with request identifiers.
- **Testing Suite.** Add unit tests for validators, password hashing, and OTP helpers, integration tests for the authentication routes using supertest against an ephemeral PostgreSQL, and Playwright smoke tests for the frontend happy path.
- **Deployment Story.** Add a Dockerfile, a docker-compose file for local development with PostgreSQL, a GitHub Actions workflow for lint and tests, and a `.env.example` listing every environment variable.
- **Performance Optimisation.** Profile the recommendation endpoints, add caching for repeated predictions, and ensure prediction responses stay under the two second target.

- **Admin Panel.** Build an admin interface for user management, system monitoring, and model performance metrics.
- **Accessibility.** Add alt text on images, ensure focus states are visible, and verify colour contrast meets WCAG AA standards.
- **Documentation.** Update the README with run instructions for backend, frontend, and the prediction service. Add module level READMEs for the backend, frontend, machine learning, and dataset folders.

5.3 Technical Deliverables

The project at the mid term defense delivers a working authentication platform on top of which the recommendation features will be built. Specifically, the codebase contains the Node.js backend built with Express.js and Prisma on PostgreSQL, the React based web application with authentication and dashboard pages, the Prisma migration scripts for the users and OTP tables, and the prepared combined dataset that will feed the XGBoost models.

In the next phase, the project will deliver a Python based prediction service backed by trained XGBoost models for geographic recommendation, tech tier classification, camera preference classification, score regression, brand loyalty classification, and the product to audience pipeline. The Node.js backend will expose the corresponding recommendation endpoints. An extended PostgreSQL schema will store user profiles, customer preferences, phone specifications, and recommendation history. A model integration layer will ensure that models load correctly and deliver predictions within the two second target. Comprehensive documentation including API specifications, database schema, model performance reports, and the technical project report will also be provided.

5.4 Impact and Benefits

For Retailers:

Retailers will gain data driven inventory decisions through location based category predictions revealing local preferences. Audience insights including tech tiers, camera preferences, and brand segments will enable targeted marketing campaigns. Understanding which customer segments to target reduces marketing waste, while regional brand intelligence will enable smarter stock allocation across store locations.

For Customers:

Customers will receive personalized recommendations matching their individual preferences rather than generic suggestions. Match scores and segment labels will provide

clear explanations building trust in the system. Time spent researching phones will be reduced through curated suggestions aligned with their tech expectations and camera needs, leading to more satisfying purchase decisions.

For the Mobile Retail Ecosystem:

The project modernizes mobile phone retail through data driven segmentation and intelligent recommendations. It will solve the cold start problem enabling recommendations for new phones and customers without historical data. The open source system using Node.js, Express.js, Prisma, PostgreSQL, React, and XGBoost can be extended to other product categories from electronics to automobiles, creating broader impact across retail sectors.

References

- [1] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Brij Kishore Pandey. The cold start problem in recommender systems, 2022. URL <https://www.freecodecamp.org/news/cold-start-problem-in-recommender-systems/>. Accessed: 2024-05-20.
- [3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [4] Bhavesh Singh. Customer segmentation recommendation. <https://github.com/bhaveshsingh0206/customer-segmentation-recommendation>, 2020. Accessed: 2026-02-26.
- [5] Peng Qi, Xiaolong Zhang, and Zhenyu Wang. An xgboost recommendation algorithm based on collaborative filtering. In *2018 IEEE 9th International Conference on Software Engineering and Service Science (ICSESS)*, pages 138–141, 2018. doi: 10.1109/ICSESS.2018.8663849.
- [6] Kavitha Ayyappan et al. Bidirectional lstm with xgboost for customer purchase intent prediction. *International Journal of Intelligent Systems and Applications in Engineering*, 11(4):284–293, 2023.
- [7] dilkushsingh. Smartphones dataset (august 2024). <https://www.kaggle.com/datasets/dilkushsingh/smartphones-dataset-up-to-july24>, 2024. Accessed: 2026-02-27.
- [8] waqi786. Mobile sales dataset. <https://www.kaggle.com/datasets/waqi786/mobile-sales-dataset>, 2024. Accessed: 2026-02-27.
- [9] NanoReview. Phone list by antutu rating. <https://nanoreview.net/en/phone-list/antutu-rating>, 2026. Accessed: 2026-02-27.